

Vector Haptics Firmware

Quickstart Guide

Feb 1, 2025

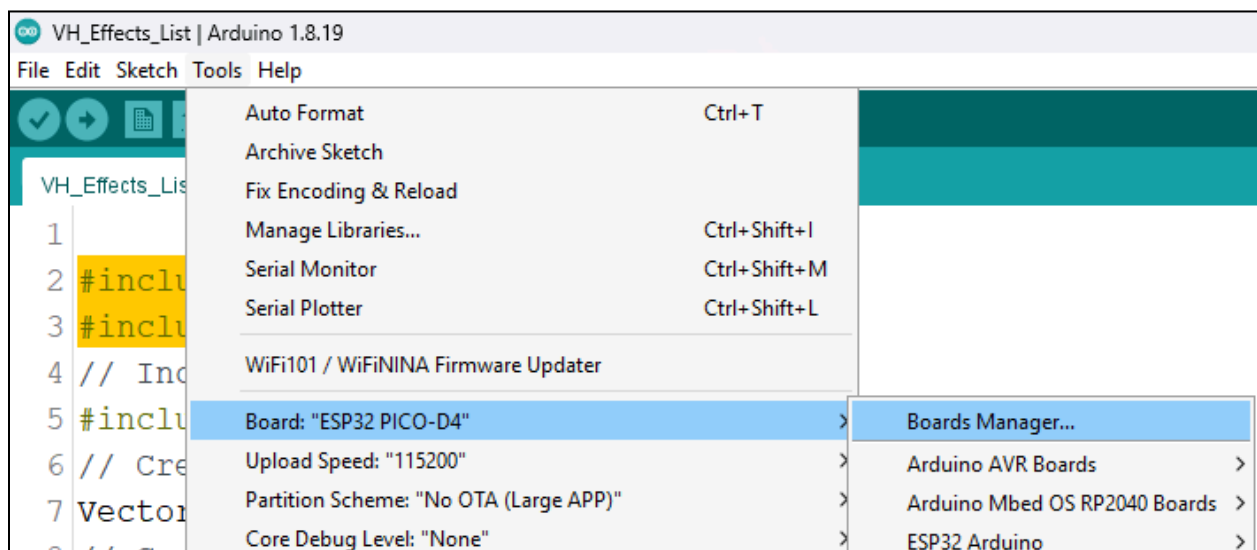
Overview: This guide helps new users quickly set up and use a Vector Haptics (VH) device to create haptic effects. For more detailed examples and instructions, please refer to the specific library documentation in the TITAN Haptic Developer Program.

Device and Firmware Setup

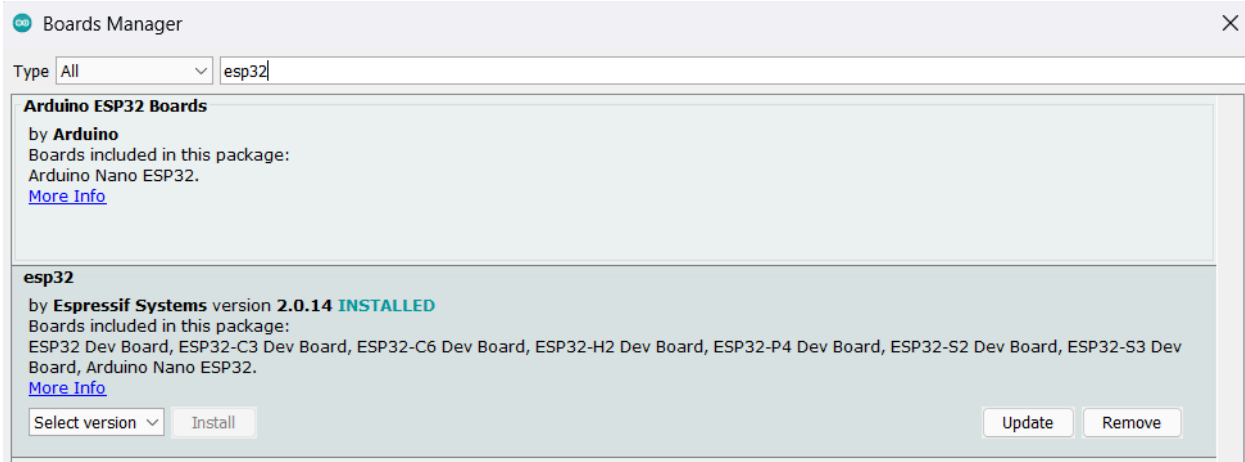
Setting up with Arduino IDE

Setting up TITAN Core Board

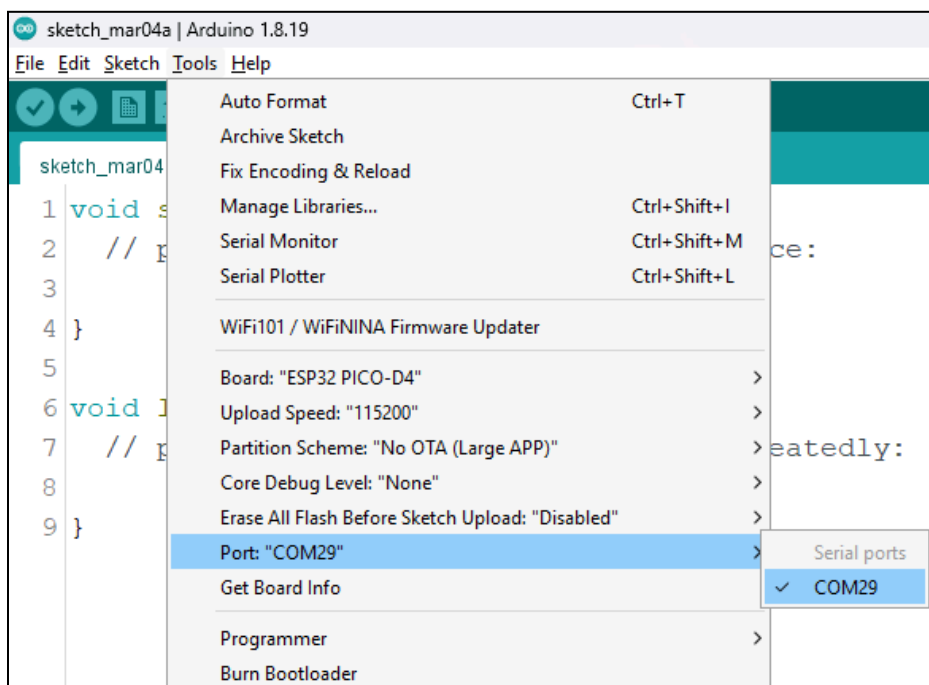
1. Ensure you have Arduino **Legacy (1.8.x version)**:
 - a. Download Arduino Legacy through [Arduino Website Link](#)
2. Launch Arduino Legacy IDE
3. Ensure you have the correct board type selected "ESP32 PICO-D4":
 - a. Tools→Board→Board Manager



- b. Search for ESP32 by Espressif Systems and Install ESP32 **version 2.0.14**

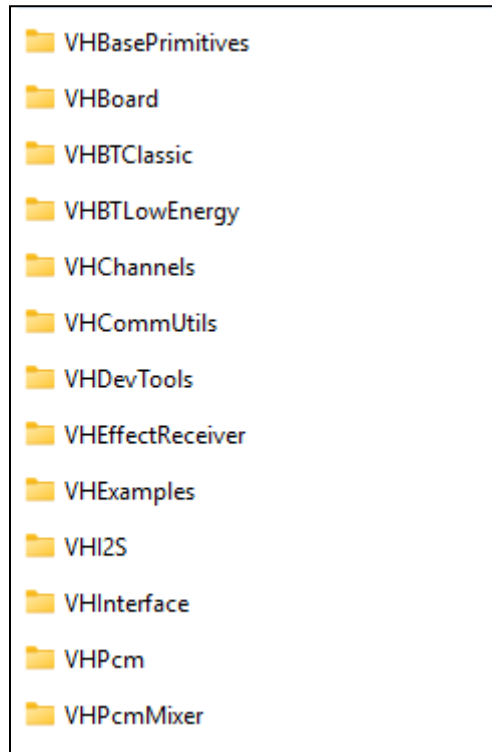


- c. Select Tools → Board → ESP32 Arduino → ESP32 PICO-D4
4. Plug in the TITAN Core to the laptop using the USB cable and select the correct settings before flashing.
 - a. Choose correct COM port for the device
 - b. Tools → Board → ESP 32 PICO-D4
 - c. Tools → Upload Speed → 115200
 - d. Tools → Port → *Port connected to your device*
 - e. Tools → Partition Scheme → No OTA (Large APP)
 - f. Verify the settings against the figure below

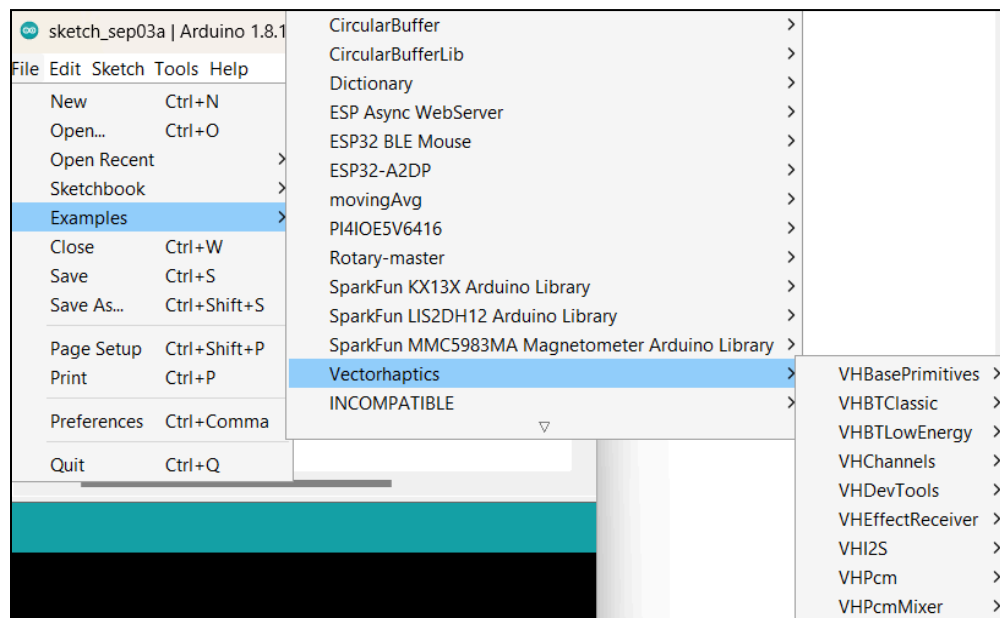


Installing Vector Haptics (VH) Library

1. Download the Vector Haptics Library from the TITAN Haptics Developer Program
2. Extract the zip file at ...\Arduino\libraries.

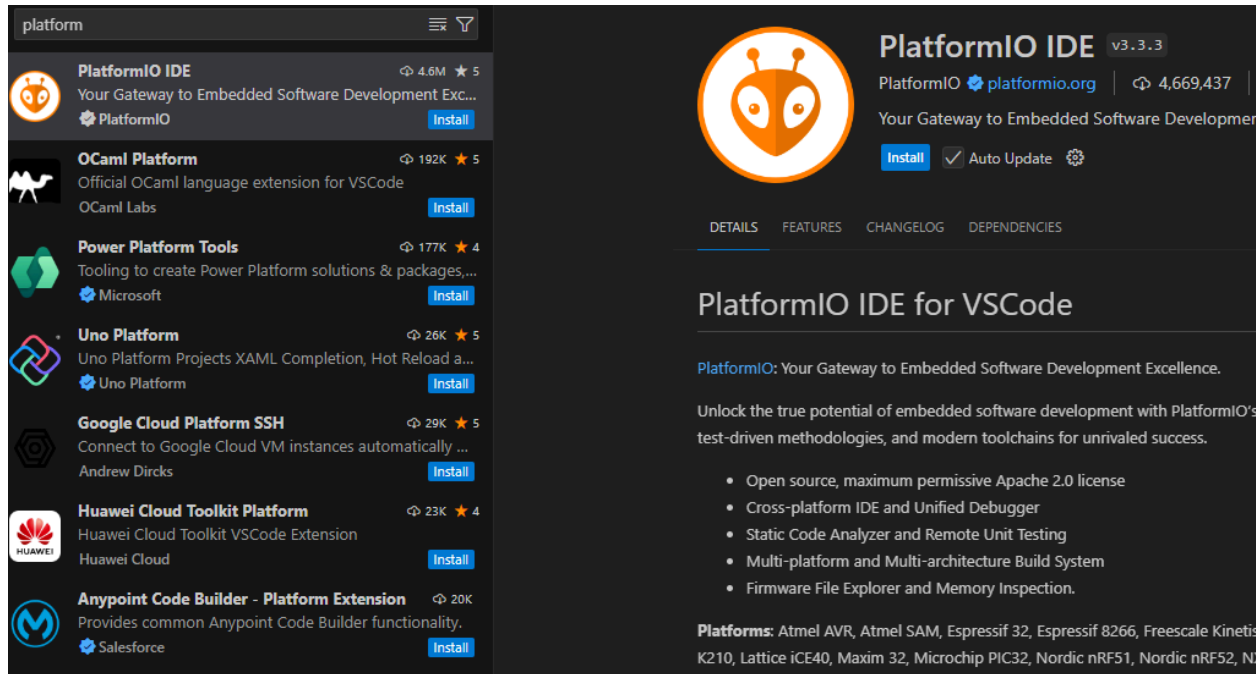


3. Restart Arduino, examples are under File → Examples → Vectorhaptics → VHxxx



Setting up with Platform IO

1. Ensure you have Visual Studio Code or download [with this link](#).
2. In the Visual Studio Code Extensions Tab, install PlatformIO Extension.



3. Watch [this video tutorial](#) on how to set up PlatformIO.
4. Create a new project with the Board set to "ESP32 Pico Kit (Espressif)" and the Framework as "Arduino".
5. Download the required VH library zip file from Titan [website](#).
6. Extract the required library folders (e.g. VHCore.zip, VHChannels.zip, etc) and copy them into the "lib" folder
7. Set up your platformio.ini (Platform IO Configuration File) as below:

```
C/C++
[platformio]
default_envs = ESP32_PICO_MINI

[env:ESP32_PICO_MINI]
platform = espressif32
board=esp32dev
platform_packages = framework-arduinoespressif32 @
https://github.com/espressif/arduino-esp32.git#2.0.14
framework = arduino
```

```
monitor_speed = 115200
upload_speed=512000
build_src_filter=+<*> -<.git/> -<.svn/> -<doc/>
board_build.arduino.upstream_packages = no

build_flags =
  -Llib/VHBoard/src/esp32
  -lVHBoard
  -Llib/VHCommUtils/src/esp32
  -lVHCommUtils
  -Llib/VHBasePrimitives/src/esp32
  -lVHBasePrimitives
  -Llib/VHChannels/src/esp32
  -lVHChannels
  -Llib/VHEffectReceiver/src/esp32
  -lVHEffectReceiver
  -Llib/VHDevTools/src/esp32
  -lVHDevTools

lib_ldf_mode = deep+
```

Note: The build_flags depend on the libraries included in the lib folder. For details on specific libraries, please refer to the Vector Haptics Library Documentations.

8. Write your example code in the src/main.ino file.

Note: Must include the Arduino header in main.cpp: #include <Arduino.h>

9. In your main.cpp, copy and paste [Example Code](#) available in this document.
10. Build and upload the project

Primitives Overview

Primitives are basic haptic effects used to create unique sensations by controlling parameters like intensity, duration, and waveform properties. They serve as building blocks for composing complex haptic experiences. Below are the 4 types of basic primitives: pulse, tick, vibrate, pause.

Types of Primitives

Pulse

Description: A single, soft impulse used for pulses, bumps, wobbles, and motion forces.

Parameters:

	Type	Range	Default
Intensity	Float	0 to 1	0.7
Duration	Float	In ms (1- 200 ms)	Function of F0
Sharpness	Float	0 (smooth) to 1 (sharp), Optional	0.5

Syntax:

C/C++

```
PULSE(1, 10, 0.8); //intensity = 1.0, duration = 10ms, sharpness = 0.8
PULSE(0.5, 20); //intensity = 0.5, duration = 20ms, sharpness = 0.5 [default]
```

Tick

Description: A single, sharp impulse used for clicks, textures, taps, and impacts.

Parameters:

	Type	Range	Default
Intensity	Float	0 to 1	0.7
Duration	Float	In ms (1- 200 ms)	Function of F0
Sharpness	Float	0 (smooth) to 1 (sharp), Optional	1.0

Syntax:

C/C++

```
TICK(1, 10, 0.8); //intensity = 1.0, duration = 10ms, sharpness = 0.8
TICK(0.5, 20); //intensity = 0.5, duration = 20ms, sharpness = 1
```

Vibrate

Description: A continuous vibration with customizable frequency, intensity, sharpness, and waveform fullness. Used for alerts, textures, and other sustained effects.

Parameters:

	Type	Range	Default
Frequency	Float	0 - 1000 Hz	F0
Intensity	Float	0 to 1	0.7
Duration	Float	In ms (1 - 10000 ms)	Function of F0
Sharpness	Float	0 (smooth) to 1 (sharp)	0

Syntax:

C/C++

```
VIBRATE(100, 1, 1000, 1); //frequency = 100hz, intensity = 1.0, duration = 1000ms, sharpness = 1

VIBRATE(250, 0.5, 250, 0.8); //frequency = 250hz, intensity = 0.5, duration = 250ms, sharpness = 0.8
```

Pause

Description: Pauses all haptic output for a specified duration.

Parameters:

	Type	Range	Default
Duration	Float	In ms (1 - 10000 ms)	500 ms

Syntax:

C/C++

```
PAUSE(1000) ; // duration = 1000ms
```

Example Code

C/C++

```
#include "Esp32PicoMini.h"
#include "VectorHaptics.h"
#include <Arduino.h>
#include <VHBasePrimitives.h>
#include <VHEffectReceiver.h>

// Defining the VectorHaptics and VHBasePrimitives objects to access the haptic
// channel and base primitives.
VectorHaptics<Esp32PicoMini> vh;
VHBasePrimitives bp;
VHEffectReceiver effRec;

void setup()
{
    VHChannel chn1(1, 25);
    chn1.setPrimitives(PULSE|TICK|VIBRATE);
    vh.setChannel(&chn1); // Adding the channel to the board.
    effRec.InitSerial();
    vh.log(ERR_LOG|WAR_LOG|INF_LOG);
    vh.Init(&bp,&effRec); // Initializing the VHBasePrimitives object.
    xTaskCreate(LoopDriver, "LoopDriver", 8000, NULL, configMAX_PRIORITIES, NULL);
}

void loop()
{
    vh.EffectDriver();
}
```

```
void LoopDriver(void *param)
{
    while (true)
    {
        // Playing basic primitives in a loop.
        vh.play(VIBRATE(100, 1, 500, 1));
        vh.play(PAUSE(500));
        vh.play(PULSE(1, 10));
        vh.play(PAUSE(1000));
        vh.play(TICK(1, 10));
        vh.play(PAUSE(5000));
        vh.play(VIBRATE(200, 1, 1000, 1));
        vh.play(PAUSE(2000));
        delay(10);
    }
}
```

More Example codes can be found on Arduino IDE

- File → Examples → Vectorhaptics → VHBasePrimitives → VH_basic_effects